

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..14

TROUBLESHOOTING

 Copy as Markdown

# Troubleshoot Apple sign-in

This page helps you identify common Sign in with Apple failures, explains why they occur, and shows how to resolve them. For Apple's full list of OAuth error codes, see [TN3107: Resolving Sign in with Apple response errors](#).

## Setup checklist

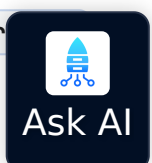
Go through this before investigating a specific error. Most problems come from a missed step.

### Apple Developer Portal

- ☐ Enable **Sign in with Apple** on your App ID at [Certificates, Identifiers & Profiles](#).
- ☐ Create a **Service ID** for OAuth.
- ☐ On the Service ID, check **Sign in with Apple**, click **Configure**, and select your **Primary App ID**.
- ☐ Add your **Domains and Subdomains** (e.g. `example.com`) and **Return URLs** on the Service ID.
- ☐ Confirm the **return URL** on the Service ID uses `https://` (not `http://` or `localhost`).
- ☐ Create a **Sign in with Apple key** and download the `.p8` file.

### Server

- ☐ Add `serverpod_auth_idp_server` to your server's `pubspec.yaml`.
- ☐ Add the Apple credentials to `config/passwords.yaml` with the raw `.p8` file contents (not a pre-generated JWT).
- ☐ Double-check the `.p8` key is indented consistently under `appleKey: |`.
- ☐ Add `AppleIdpConfigFromPasswords()` to `identityProviderBuilders` in `serverpod_auth_idp_server`.
- ☐ Call `pod.configureAppleIdpRoutes(...)` on the server before the pod starts.
- ☐ Create an `AppleIdpEndpoint` file in `lib/src/auth/`.



- ☐ Start the server with `serverpod start`, then create and apply the migration (press **M**).

## Client

- ☐ Add `serverpod_auth_idp_flutter` to your Flutter app's `pubspec.yaml`.
- ☐ Add `client.auth.initializeAppleSignIn()` after `client.auth.initialize()` inside `initializeClient()` in your Flutter app's `lib/client.dart`.
- ☐ Add **Sign in with Apple** under Signing & Capabilities in Xcode (*iOS/macOS only*).
- ☐ Add the **Apple JS SDK** script to `web/index.html` (*web only*).
- ☐ Pass `APPLE_SERVICE_IDENTIFIER` and `APPLE_REDIRECT_URI` via `--dart-define`. Initialization throws an `ArgumentError` without them, on every platform.
- ☐ Add the `signinwithapple` intent filter to `AndroidManifest.xml` (*Android only*).
- ☐ Add **Apple's mail servers** to your SPF record if you email users who might use Hide My Email.

## Sign-in fails with `invalid_client` every time

**Problem:** Every authentication attempt gives an `invalid_client` error from Apple.

**Cause:** The `appleKey` value in `passwords.yaml` is not indented correctly. The key gets corrupted during parsing. The server starts without error, but Apple sees an invalid signature.

**Resolution:** Paste the raw `.p8` key under `appleKey` with consistent indentation. All lines of the key must line up with the one that starts `-----BEGIN PRIVATE KEY-----`. For example:

```
appleKey: |
  -----BEGIN PRIVATE KEY-----
  MIGTAgEAMBMGBYqGSM49AgEGCCqGSM49AwEHBHkwdwIBAQQg...
  -----END PRIVATE KEY-----
```

Alternatively, set `appleKey` via the `SERVERPOD_PASSWORD_appleKey` environment variable to avoid YAML indentation entirely.

## Sign-in starts failing with `invalid_client` after months of success

**Problem:** Sign-in was working for months, then suddenly fails with `invalid_client` and you haven't changed code.

**Cause:** The `appleKey` value holds a pre-generated client secret JWT, not the raw `.p8` key. Apple makes JWTs expire after six months. When it expires, all sign-ins fail.

**Resolution:** Replace any JWT in `appleKey` with the raw `.p8` private key (include the full header and footer). Serverpod will create fresh short-lived JWTs automatically. No need to handle JWTs yourself. See

Creating a client secret.

## Sign-in fails with `invalid_grant`

**Problem:** Authentication fails with an `invalid_grant` error from Apple.

**Cause:** Apple's authorization codes are single-use and expire after five minutes. This error occurs when:

- The authorization code was already exchanged (e.g. the request was retried after a network failure).
- The server clock is significantly out of sync, causing the client secret JWT to appear expired before Apple processes it.

**Resolution:**

- Do not retry requests that carry an Apple authorization code. If the flow fails, restart it from the beginning.
- Ensure your server's system clock is synchronized via NTP. A drift of more than a few seconds will cause JWT validation to fail on Apple's side.

## Wrong identifier passed for web or Android sign-in

**Problem:** Sign-in on Android or web fails immediately, or Apple returns `invalid_client` / `invalid_request` even though credentials look correct.

**Cause:** There are two separate identifiers in Apple's system and they are easy to mix up:

- **App ID** (`bundleIdentifier`): the bundle identifier of your iOS/macOS app (e.g. `com.example.app`). Used for native Apple platform sign-in only.
- **Services ID** (`serviceIdentifier`): a separate identifier you create in the Apple Developer Portal specifically for web and Android OAuth (e.g. `com.example.service`). This acts as the OAuth client ID.

Passing the App ID bundle identifier where the Services ID is expected will cause Apple to reject the request.

**Resolution:** Check `passwords.yaml` and confirm:

- `appleServiceIdentifier` is set to your **Services ID** (the one created under Identifiers → Services IDs).
- `appleBundleIdentifier` is set to your **App ID** bundle identifier.

If you use `--dart-define`, confirm `APPLE_SERVICE_IDENTIFIER` is the Services ID, not the bundle ID.

## Sign-in hangs on Android

**Problem:** The OAuth flow opens a browser, but never returns to the app. Sign-in seems to finish but the app doesn't get the callback.

**Cause:** The `signinwithapple` URI scheme isn't registered in `AndroidManifest.xml`, so Android drops the callback.

**Resolution:** Add this activity to `AndroidManifest.xml`:

```
<activity
  android:name="com.aboutyou.dart_packages.sign_in_with_apple.SignInWithAppleCallback"
  android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <category android:name="android.intent.category.BROWSABLE" />
    <data android:scheme="signinwithapple" />
    <data android:path="callback" />
  </intent-filter>
</activity>
```

## Server crashes on first Apple sign-in with "no such table"

**Problem:** The server builds and starts, but crashes when a user tries Apple sign-in. The error cites a missing table (like `serverpod_auth_idp_apple_account`).

**Cause:** The database migration that creates the provider's tables was never created or applied.

**Resolution:** In the running `serverpod start` terminal, press **M** to create and apply the migration.

## Apple rejects the redirect URI with `invalid_request`

**Problem:** The web OAuth flow fails with `invalid_request` and Apple's error page says the redirect URI is invalid.

**Cause:** You're using HTTP instead of HTTPS for the redirect. Apple requires HTTPS and does not allow `localhost`.

**Resolution:** Always use an HTTPS URL for your redirect. For local development, run your server behind an HTTPS tunnel. Register the tunneled `https://` URL as your return URL in Apple's Developer Portal, and update `appleRedirectUri` in `passwords.yaml` to match.

## Emails aren't delivered to some users

**Problem:** Transactional emails (password resets, notifications) work for most people, but some never receive them.

**Cause:** Some users chose Apple's "Hide My Email" during sign-in. Mail to relay addresses like `@privaterelay.appleid.com` fails if your domain's SPF record doesn't include Apple's mail servers.

**Resolution:** Add Apple's mail servers to your SPF record. See [Configure private email relay service](#) for SPF settings and instructions.

## User email is `null` after sign-in

**Problem:** The user's email is missing or `null` after sign-in, or it's present on first sign-in but missing after that.

**Cause:** Apple sends the email address and name only once, during the initial authorization. Later sign-ins carry only the stable `sub` identifier. Serverpod stores both values on the `AppleAccount` row when it first creates the account. The usual reason they are missing is that the first authorization never completed on your server, for example because the endpoint was added afterwards.

**Resolution:** The module does not backfill these fields on later sign-ins. Ask the user for their email in your app. If the user removes your app in their Apple ID settings (**Sign in with Apple > Stop Using Apple ID**) and signs in again, Apple resends the values, but the module keeps the old account row. Sign-in itself is unaffected either way, since accounts are keyed by Apple's stable identifier, not the email. See [Authenticating users with Sign in with Apple](#).

## iOS sign-in prompt doesn't show

**Problem:** Tapping the Sign in with Apple button on iOS has no effect, or the Apple authentication UI never appears.

**Cause:** The App ID isn't set up with the Sign in with Apple capability, or your Xcode project isn't using that App ID.

**Resolution:**

1. In [Certificates, Identifiers & Profiles](#), find your App ID. Check that **Sign in with Apple** is enabled under Capabilities.
2. In Xcode, select your target, open **Signing & Capabilities**, and check that **Sign in with Apple** is listed. If not, click + **Capability** to add it.
3. Download and install your new provisioning profile if needed.

## Sign-in does not work in the iOS Simulator

**Problem:** Sign in with Apple silently fails or the native authentication sheet does not appear when running in the iOS Simulator, but works fine on a physical device.

**Cause:** Some Simulator versions do not fully support the native Sign in with Apple flow. This is a known Simulator limitation, not a code or configuration issue.

**Resolution:** Test on a physical device to confirm the problem is Simulator-specific. If sign-in works on a real device, no changes are needed.

## Web sign-in fails with `TypeError: type ... is not a subtype of type 'JSObject'`

**Problem:** Clicking the Apple button on web throws a `TypeError` mentioning `JSObject` or a minified type like `minified:CM`.

**Cause:** The Apple JS SDK is not loaded. The `sign_in_with_apple` package calls `AppleID.auth.init()` on the page, but that function only exists after Apple's script is loaded in the HTML.

**Resolution:** Add the Apple JS SDK to your Flutter app's `web/index.html` inside the `<head>` tag:

```
<script type="text/javascript" src="https://appleid.cdn-apple.com/appleauth/static/jsapi/appleid/1/en_US/appleid.auth.js"
crossorigin="anonymous"></script>
```

## macOS sign-in shows "Sign Up Not Completed"

**Problem:** The native Sign in with Apple sheet appears on macOS, but immediately shows "Sign Up Not Completed" without completing authentication.

**Cause:** This is almost always a signing or entitlements mismatch on the macOS target. Sign in with Apple needs the bundle ID, Team ID, and entitlements to line up with an App ID that has the capability enabled.

**Resolution:** Check, in this order:

1. In Xcode, open the macOS target's **Signing & Capabilities** tab and confirm that **Sign in with Apple** is listed. If not, click + **Capability** to add it.
2. Confirm the macOS bundle ID matches the App ID that has Sign in with Apple enabled in the Apple Developer Portal, and that the signing **Team** matches the same Apple Developer account.
3. If your macOS app is sandboxed, make sure the sandbox entitlements include `com.apple.security.network.client`. Without outbound network access, the request to Apple's servers fails silently.

4. Regenerate or re-download the provisioning profile after any of the above changes so the new entitlements are picked up.

## User stays signed in after removing Apple access

**Problem:** A user removes your app from Apple ID settings (`Settings > [your name] > Sign-In & Security > Sign in with Apple > Stop Using Apple ID`) but is still logged in to your app.

**Cause:** Apple's revocation notification never reaches your server. Once it does, Serverpod revokes the Apple authorization and the tokens it issued through Apple sign-in automatically.

**Resolution:** Check that `pod.configureAppleIdpRoutes()` registers a `revokedNotificationRoutePath`, that the route's public HTTPS URL is registered as the server-to-server notification endpoint in the Apple Developer Portal, and that the URL is reachable from the internet. See [Processing changes for Sign in with Apple accounts](#) for how the notification works.

## Related

- [Setup](#): configure Sign in with Apple on the server and in your app.
- [Customizations](#): configuration options and sign-in UI customization.
- [UI components](#): the sign-in widgets and how to compose them.